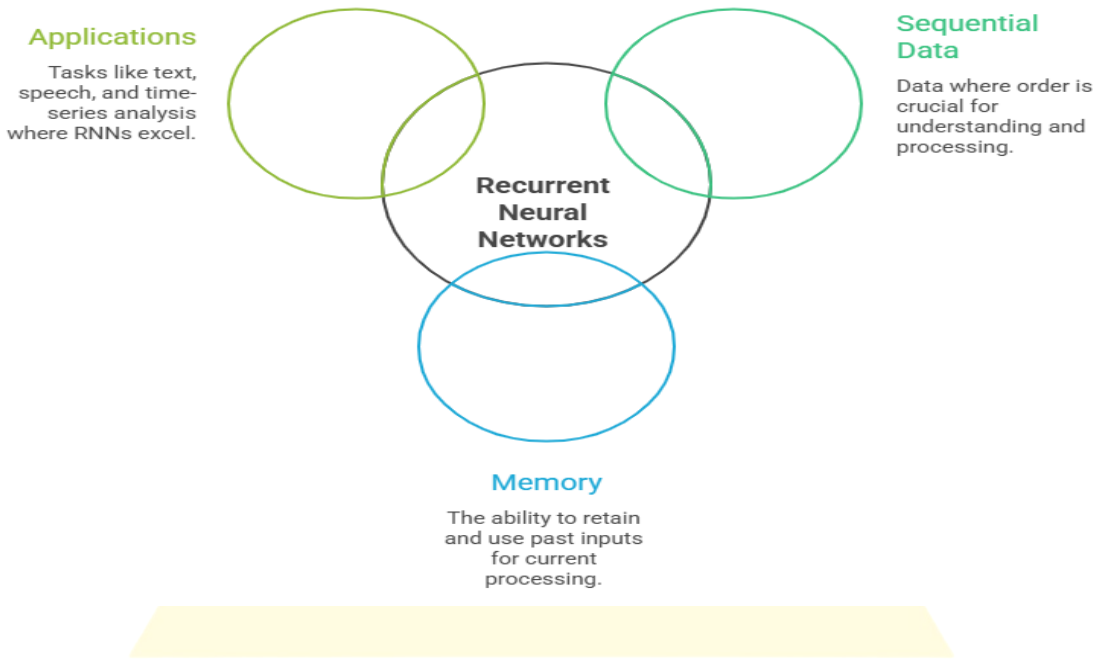
Fundamentals of Recurrent Neural Networks (RNNs)

Overview

- **What are RNNs?**

  Recurrent Neural Networks (RNNs) are a class of neural networks designed to work with sequential data (data where order matters).

  Unlike feedforward networks (MLP, CNN), RNNs have a memory of previous inputs, making them suitable for tasks like text, speech, and time-series.



- **Why sequence modeling?**

  Many real-world problems involve temporal or sequential dependencies:

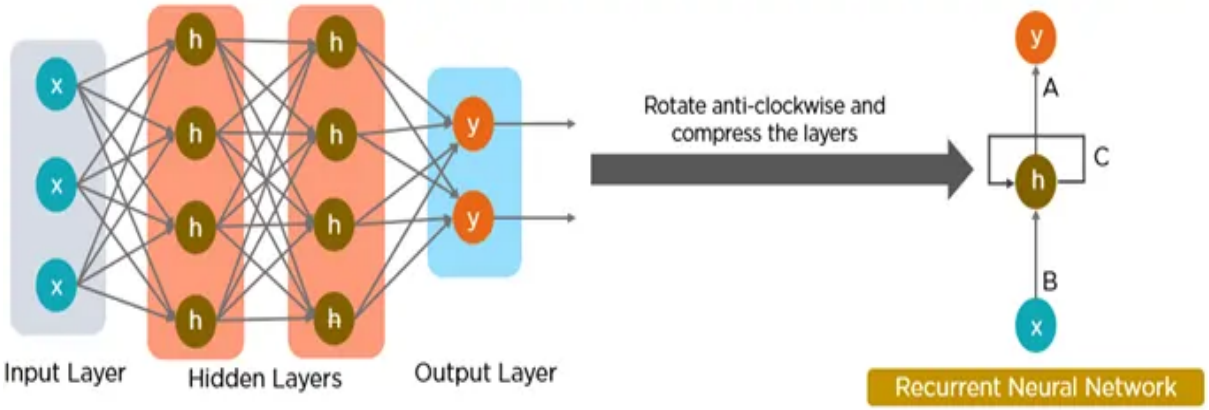
  - Predicting the next word in a sentence (NLP).
  - Understanding speech audio signals.

- Forecasting stock prices or weather.
  Traditional models treat inputs as independent, but RNNs capture order and context.

- **Analogy**
  Reading a book → You don't forget previous words while reading the next one. Similarly, RNNs carry past information to help predict the future.

RNN Architecture

An RNN processes sequences step by step, maintaining a hidden state that acts like memory.



- **Components:**
  - Input (x_t): The input at time step *t*.
  - Hidden State (h_t): Memory of past + current input.
  - Output (y_t): Prediction at time step *t*.

- Recurrence relation (mathematical form):

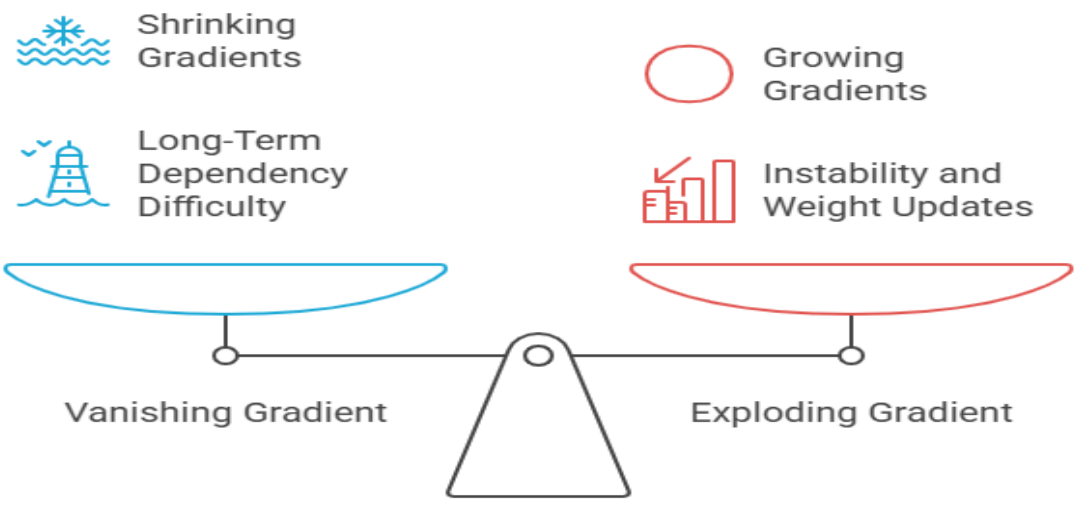
$$h_t = f(W x_t + U h_{t-1} + b)$$

Where:

- W = input weight matrix
- U = hidden state weight matrix
- b = bias
- f = activation function (often tanh or ReLU)
- Unrolling an RNN:
  Imagine an RNN across a sentence:
    - At word 1 → hidden state updates.
    - At word 2 → hidden state depends on word 2 + previous state.
    - Repeat until the sequence ends.

This allows information to flow through time.

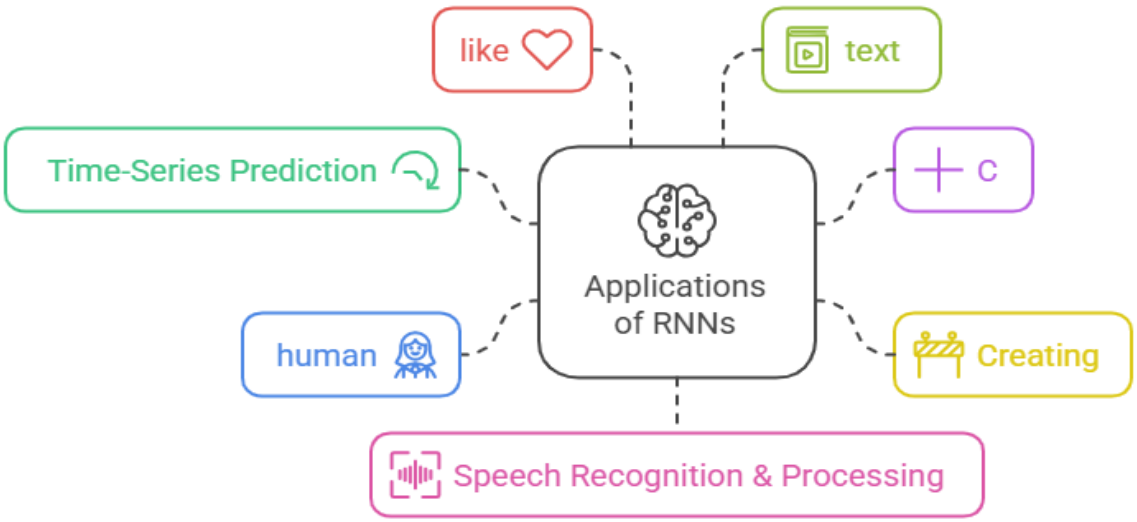
Limitations of RNNs



While powerful, vanilla RNNs have key challenges:

1. **Vanishing Gradient Problem**
    - During backpropagation, gradients shrink as they are multiplied across many time steps.
    - Makes learning long-term dependencies very hard.
    - Example: Predicting the subject of a sentence when it appears far from the verb.
2. **Exploding Gradient Problem**
    - Sometimes gradients become extremely large during training.
    - Causes instability, weight updates blow up.
    - Usually controlled with gradient clipping.
3. **Short-Term Memory**
    - RNNs remember only a few past steps effectively.
    - Struggle with very long sequences (e.g., full book context).
    - Solution → LSTM and GRU architectures.

Applications of RNNs



1. **Time-Series Prediction**
    - Stock price forecasting.
    - Weather prediction.
    - Predicting demand in the supply chain.
2. **Text Generation (NLP)**
    - Generating sentences character by character.
    - Autocomplete in search engines.
    - Language modeling.
3. **Speech Recognition & Processing**
    - Converting speech → text (Google Voice, Siri).
    - Voice assistants understanding commands.

    - Music generation and audio analysis.

Key Takeaways

- RNNs are designed for sequential data.
- They have a hidden state (memory) that connects the past with the present.
- Great for time-series, text, and speech tasks.
- Limitations: vanishing/exploding gradients and short-term memory.
- These challenges led to advanced models like LSTM and GRU.